

BLOCK 23

Controlling Loops with `continue`, `break`, and `while`

Skip records, stop early, and repeat while a condition remains true.

Normal loop

Skip one item

Stop the loop

**Repeat by
condition**

```
for record in records:
    if not record["active"]:
        continue

    if record["critical_error"]:
        break

    print("process")
```

Today's mission: control the flow without losing the plot.

BLOCK 23

Normal loop execution

A loop usually processes every item in order.

```
for record in records:  
    print(record["record_id"])  
    print("Processed")
```

Default behavior: every item gets the full loop body.

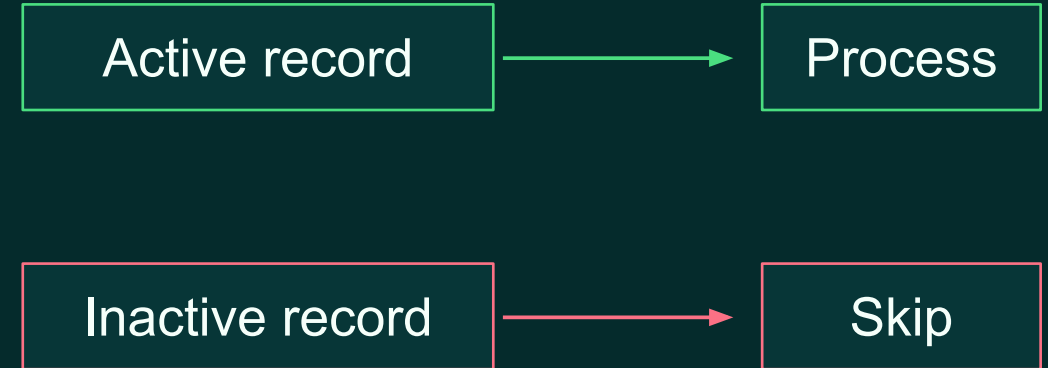


BLOCK 23

continue skips the rest of this iteration

Use it when this item should not continue into later logic.

```
for record in records:  
    if not record["active"]:  
        continue  
  
    print(f'Processing {record["record_id"]}')
```



continue does not end the loop. It moves to the next item.

BLOCK 23

continue validation pattern

Invalid records can be rejected before expensive calculations.

```
for record in records:
    if record["record_id"] == "":
        print("Skipping record with missing ID")
        continue

    print(f'Processing {record["record_id"]}')
```

Fail fast

Check basic validity first.

Skip bad records before later calculations.

This connects directly to defensive programming.

BLOCK 23

break stops the loop completely

Use it when there is no reason to keep checking items.

```
for record in records:  
    if record["error_count"] > 100:  
        print("Critical threshold reached")  
        break  
  
    print(f'Processing {record["record_id"]}')  

```

Before break

Loop continues

Break condition

Loop ends now

break exits the nearest loop.

Searching with break

Stop after finding the target record.

```
search_id = "EQ-1003"

for record in records:
    if record["record_id"] == search_id:
        print("Record found")
        break
```

Why stop?

- The goal has been met
- Extra checks waste time
- The output is clearer
- The intent is explicit

Search pattern: scan → match → report → stop.

BLOCK 23

continue vs. break

Same loop family, very different result.

```
if invalid:  
    continue  
# next item
```

continue

Skip current iteration only

```
if critical:  
    break  
# loop is over
```

break

End the whole loop now

**Decision prompt: should the next item still
be examined?**

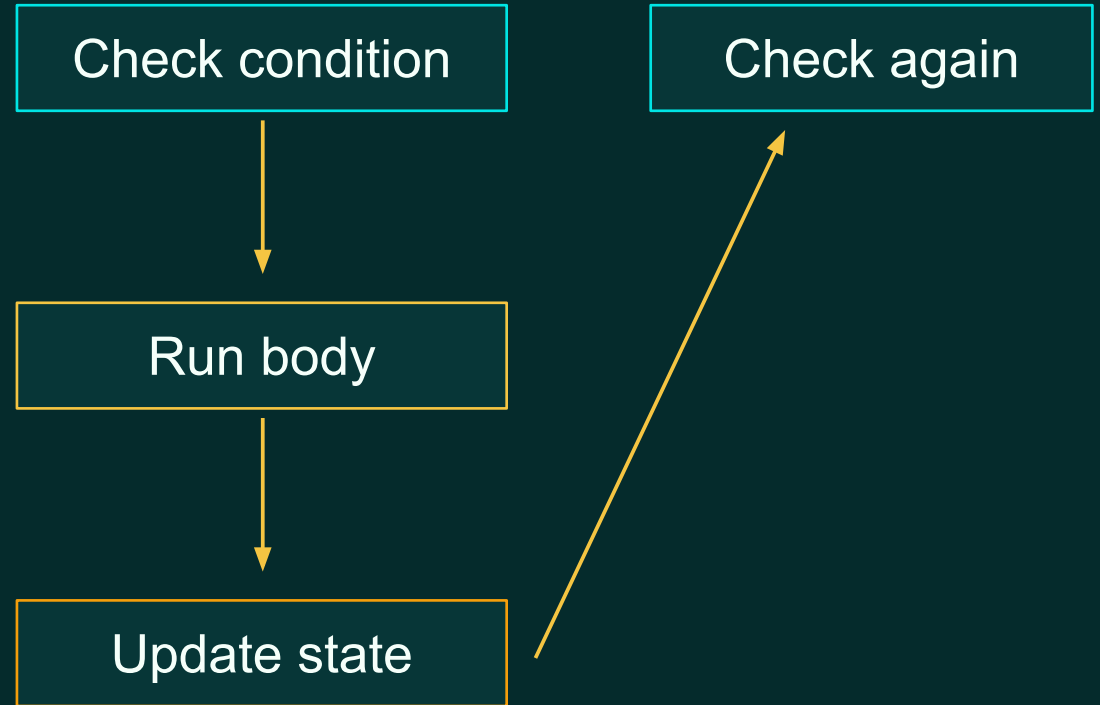
BLOCK 23

while loops repeat by condition

The loop continues as long as the condition is True.

```
count = 1

while count <= 5:
    print(count)
    count += 1
```



When condition becomes False, the loop ends.

BLOCK 23

while needs a path to stop

A missing update can create an infinite loop.

```
count = 1
```

```
while count <= 5:  
    print(count)
```

Danger

count never changes

condition stays True

loop never ends

```
count = 1
```

```
while count <= 5:  
    print(count)  
    count += 1
```

Safe rule: something in the loop should move toward the stopping condition.

Sentinel-controlled input

Repeat until the user enters a special value.

```
command = ""  
  
while command != "quit":  
    command = input("Enter command: ").strip().lower()  
  
    if command != "quit":  
        print(f"You entered: {command}")
```

Sentinel value

A special input that means
“stop.”

Here, quit ends the loop.

Useful when you do not know how many inputs there will be.

for versus while

Choose the loop that matches the intent.

Use for when...

- You have a collection
- You know the items to process
- You want each item once
- You use range() for known counts

Collection-driven

Use while when...

- Repetition depends on a condition
- The count is unknown
- User input controls when to stop
- State changes until done

Condition-driven

Guided exercise 1: skip invalid records

Use continue to protect later processing.

Skip records with:

- Missing ID
- Negative record count
- Inactive status

Exercise time: 7 minutes

```
for record in records:
    if record["record_id"] == "":
        continue

    if record["record_count"] < 0:
        continue

    if record["status"] != "active":
        continue

    print("Processing", record["record_id"])
```

BLOCK 23

Guided exercise 2: search and stop

Find EQ-1004, print it, then break.

```
target_id = "EQ-1004"

for record in records:
    if record["record_id"] == target_id:
        print(record)
        break
```

Trace manually

- Which records are checked?
- Which one matches?
- What prints?
- What records are skipped after break?

Exercise time: 5 minutes

Guided exercise 3: critical error stop

Process records until a critical flag appears.

```
for record in records:  
    if record["critical_error"]:  
        print("Critical error found")  
        break  
  
    print("Processed", record["record_id"])
```

Critical condition

Continuing could be unsafe or misleading.

break means: stop the queue now.

Exercise time: 5 minutes

Guided exercise 4: basic while loop

Create a countdown and prove it stops.

```
count = 5

while count > 0:
    print(count)
    count -= 1

print("Done")
```

Prediction

- What prints first?
- What prints last?
- When does condition become False?
- Why does Done print once?

Exercise time: 5 minutes

BLOCK 23

Guided exercise 5: user-controlled loop

Ask for filenames until the user enters quit.

```
file_name = ""  
  
while file_name != "quit":  
    file_name = input("File name: ").strip().lower()  
  
    if file_name == "quit":  
        break  
  
    print(f"Received: {file_name}")
```

Try inputs

report.csv

DATA.JSON

quit

Exercise time: 8 minutes

BLOCK 23

Applied challenge: processing queue

Predict what happens before running the code.

```
records = [  
    {"record_id": "EQ-1001", "status": "active",  
     "critical_error": False},  
    {"record_id": "", "status": "active",  
     "critical_error": False},  
    {"record_id": "EQ-1003", "status": "inactive",  
     "critical_error": False},  
    {"record_id": "EQ-1004", "status": "active",  
     "critical_error": True},  
    {"record_id": "EQ-1005", "status": "active",  
     "critical_error": False},  
]
```

Rules

- Missing IDs are skipped
- Inactive records are skipped
- Valid active records are processed
- Critical error stops everything

Challenge time: 15 minutes

Challenge scaffold: queue logic

Combine continue and break carefully.

```
for record in records:
    if record["record_id"].strip() == "":
        print("Skipping missing ID")
        continue

    if record["status"] != "active":
        print("Skipping inactive record")
        continue

    if record["critical_error"]:
        print("Critical error: stopping process")
        break

    print(f'Processing {record["record_id"]}')
```

Order matters

Skip invalid records before processing.

Stop immediately on critical error.

EQ-1005 should never process.

BLOCK 23

Challenge expected path

Trace each record by hand.

EQ-1001

processed

blank ID

skipped

EQ-1003

skipped inactive

EQ-1004

critical → break

EQ-1005

never reached

Processed records

EQ-1001 only

**This is why prediction matters
before automation.**

BLOCK 23

Common mistakes

Loop-control bugs usually come from placement.

continue too early

```
continue  
print("never runs")
```

break too broad

```
if error_count > 0:  
    break
```

while never updates

```
while count < 5:  
    print(count)
```

Debug strategy

Print the current item. Print the decision. Print when you skip or stop.

Knowledge check

Explain these ideas in plain language.

continue

break

while

sentinel

- What happens after continue?
- What happens after break?
- What must a safe while loop include?
- When would you choose while instead of for?
- Why predict loop output before running?

Goal: control repetition intentionally.

BLOCK 23

Bridge to the next block

Loops now have enough control for real data workflows.



The pattern:

**process each record, protect the logic, and
produce a useful result.**

**Next: richer loop patterns and more automated data
processing.**